



One calm home for everything you're studying.

A student productivity & academic management workspace —
built from the HTML5, CSS3 & JavaScript syllabus.

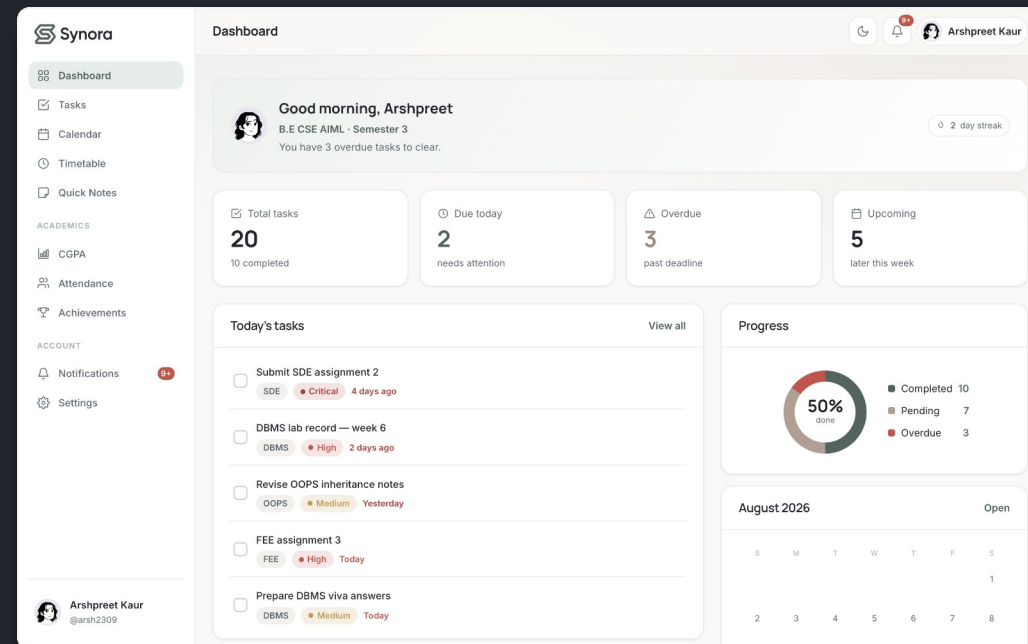
HTML5

CSS3

JavaScript

DOM

Events



The Synora dashboard — the product this deck is about.

THE PRODUCT

What is Synora?

Synora brings everything a student juggles into one calm, personal workspace — so time goes to studying, not to organising eight different apps.

Tasks

Calendar

Quick Notes

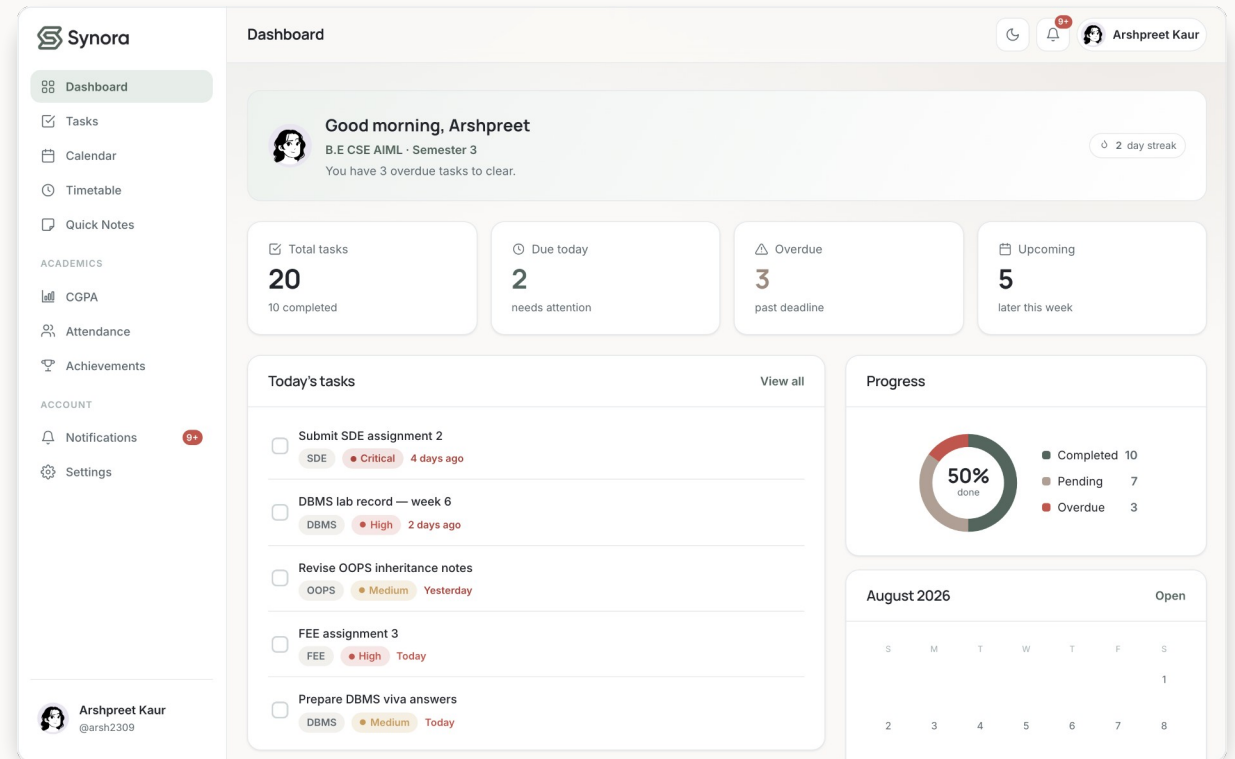
Timetable

Attendance

CGPA

Achievements

Notifications



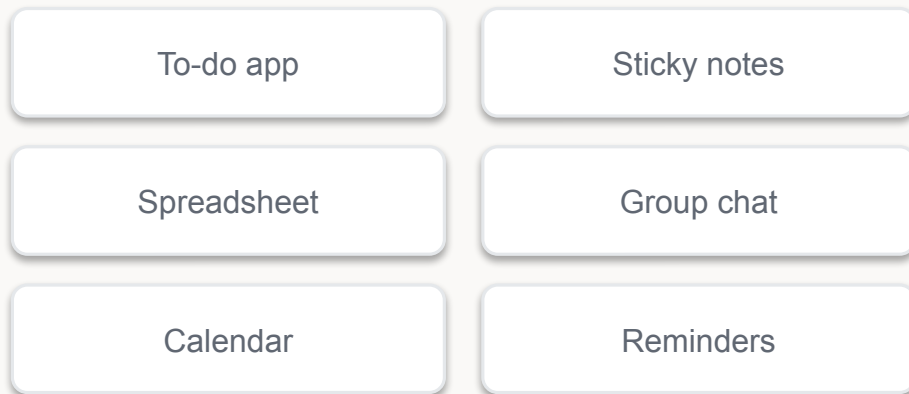
Dashboard — today's tasks, weekly progress and a mini-calendar at a glance.

One account. One quiet home for the whole semester.

THE PROBLEM

Why we built it

A student's semester is scattered across a to-do app, the class WhatsApp, a notes app, a spreadsheet for attendance and another for grades. Nothing talks to anything.



Scattered tools, constant context-switching



ONE CALM WORKSPACE

- Tasks, deadlines & notes in one place
- Attendance and CGPA that update live
- Timetable, calendar and reminders together
- Personal to each student, saved on-device

THE FOUNDATION

How Synora is built

The syllabus building blocks are the whole foundation of the product — each layer does one job.

HTML5	Structure & meaning	Semantic pages, forms, accessible controls
CSS3	Design & responsive layout	Design tokens, Flexbox, Grid, media queries
JavaScript	Logic & interactivity	State, functions, arrays, objects, loops
DOM APIs	A dynamic interface	Select, read, write, create & remove nodes
Events	Response to the user	addEventListener, delegation, validation
localStorage	Persistence	Every user's data saved in the browser

OUR THESIS

The syllabus gave us the web-development building blocks.

Synora combines those concepts into one complete, responsive student-productivity product.

Five questions, one product



The five layers combine into **Synora** — a student productivity workspace.

Building the structure

Every Synora page is written with semantic HTML5 — elements that describe what each region means, not just how it looks. The browser, assistive tech and our own code all read that structure.

<header>

<nav> aria-label

<main> ×14

<section> ×11 / aria-labelledby

<aside> ×12 sidebar

<footer>

<figure> ×3

index.html — landing structure

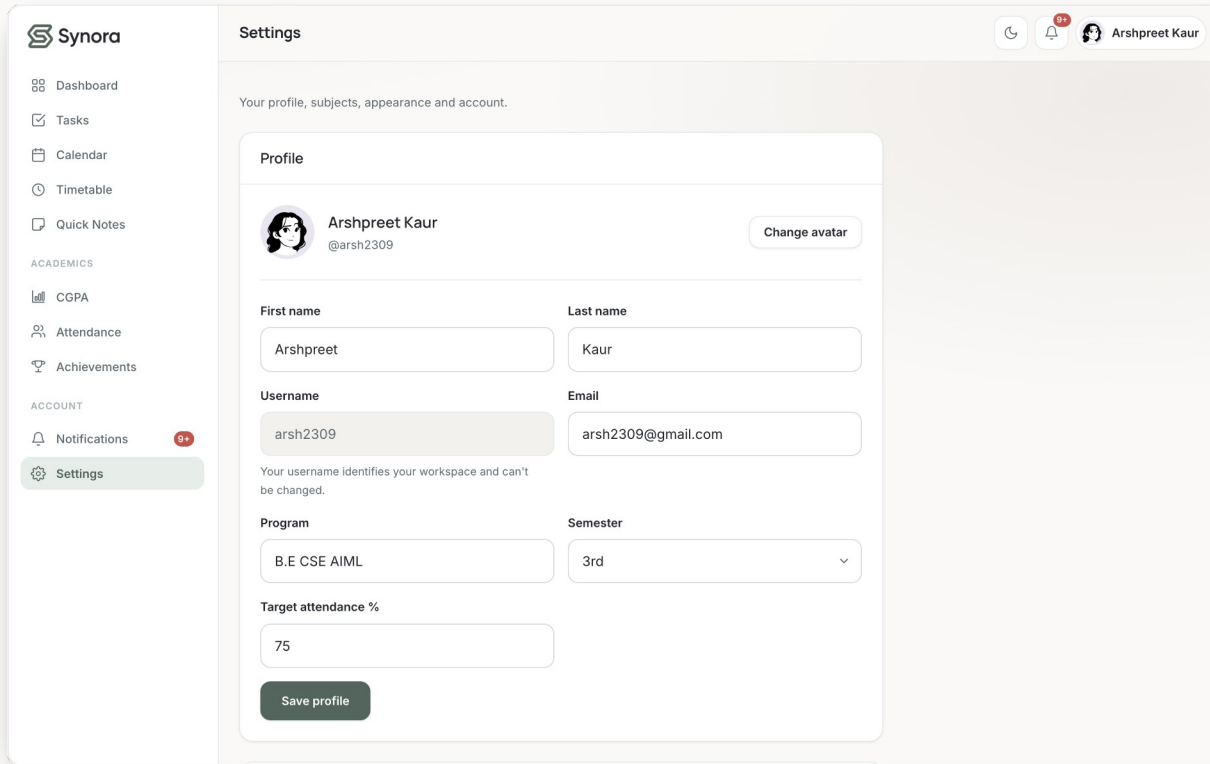
```
<header class="site-header">
  <nav aria-label="Primary"> ... </nav>
</header>

<main id="main">
  <section class="hero">...</section>
  <section aria-labelledby="features">...
  </section>
</main>

<footer class="site-footer">...</footer>
```

Real markup from index.html — trimmed.

Meaningful, accessible controls



The screenshot shows a web application interface for 'Synora' with a 'Settings' page. The page title is 'Settings' and the subtitle is 'Your profile, subjects, appearance and account.' The user is 'Arshpreet Kaur' with a profile picture and a 'Change avatar' button. The form contains the following fields:

- Profile:** A profile picture and the name 'Arshpreet Kaur' with the handle '@arsh2309'.
- First name:** A text input field containing 'Arshpreet'.
- Last name:** A text input field containing 'Kaur'.
- Username:** A text input field containing 'arsh2309' with a note: 'Your username identifies your workspace and can't be changed.'
- Email:** A text input field containing 'arsh2309@gmail.com'.
- Program:** A text input field containing 'B.E CSE AIML'.
- Semester:** A dropdown menu showing '3rd'.
- Target attendance %:** A text input field containing '75'.

A 'Save profile' button is at the bottom of the form.

Settings — every field has a real <label> and input type.

Labels for every input

16 <label> elements — clicking a label focuses its field.

ARIA where it helps

91 aria-* attributes: aria-label, aria-labelledby, aria-live regions.

Right input types

type=email, number, date, password — the keyboard and validation adapt.

Alt text on imagery

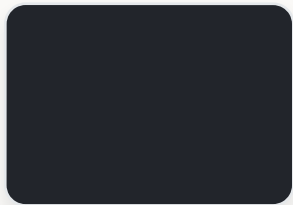
18 alt attributes; decorative art is marked aria-hidden.

Keyboard-friendly

A skip-link and a .visually-hidden pattern for screen-reader-only text.

One design system, not many pages

CSS3 custom properties define the look once — ~142 design tokens for colour, type, spacing, radius and shadow. Every card, button and form reuses them, so the whole app feels like one product.



Ink

#22252B



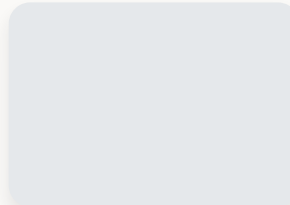
Sage

#53655C



Slate

#A3AEB1



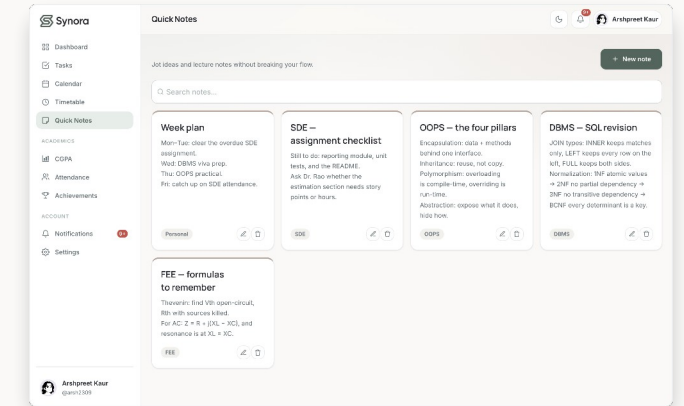
Mist

#E5E8EB



Taupe

#B09F95



Quick Notes — same cards, buttons and spacing tokens.

Typography

Manrope

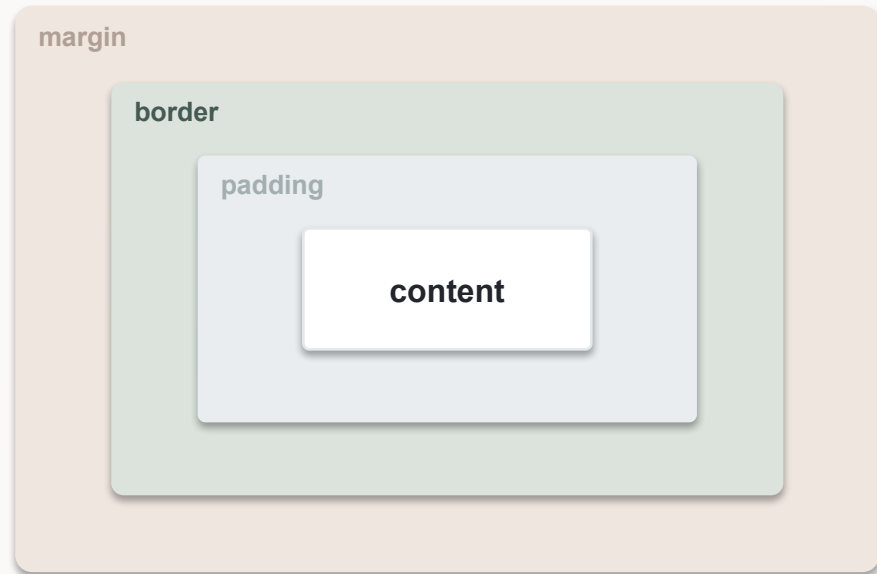
headings · Inter for body text

Reused components

`.btn` `.card` `.field` `.chip`

defined once, used everywhere

Every component is a box

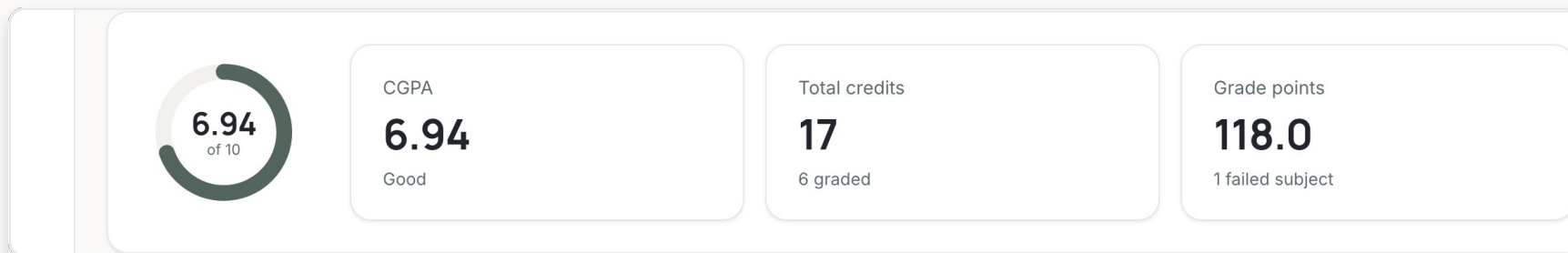


content → padding → border → margin

```
styles/app.css — a dashboard stat card

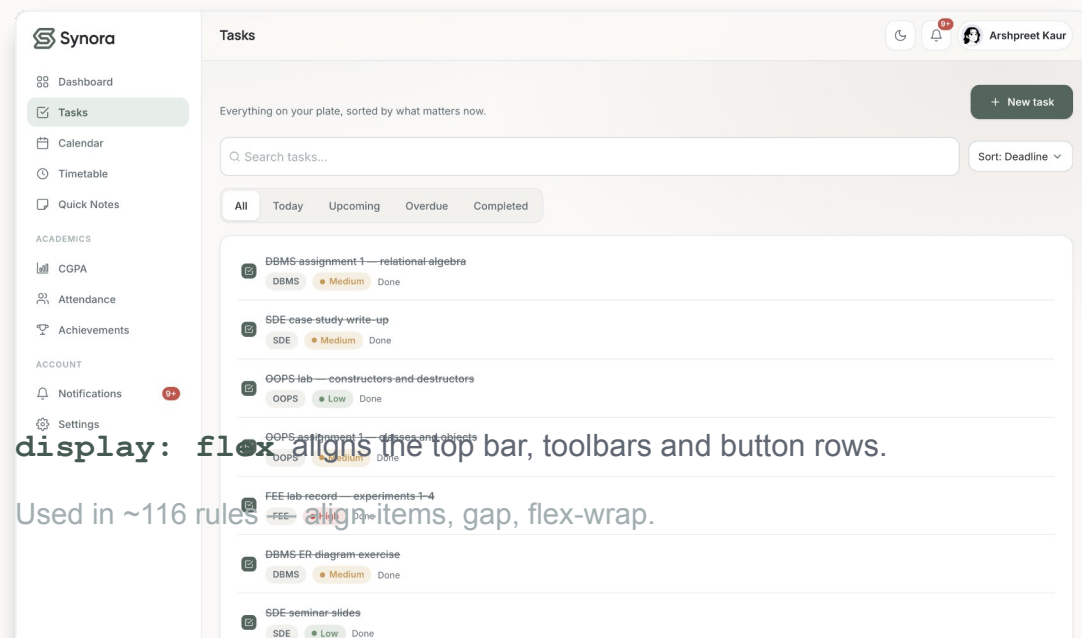
.stat-card {
  padding: 20px 24px;      /* space inside */
  background: var(--color-surface);
  border: 1px solid var(--color-border);
  border-radius: var(--radius-lg);
  box-shadow: var(--shadow-sm);
}
```

Those same tokens build every card — here, the CGPA summary row:



Flexbox for rows, Grid for structure

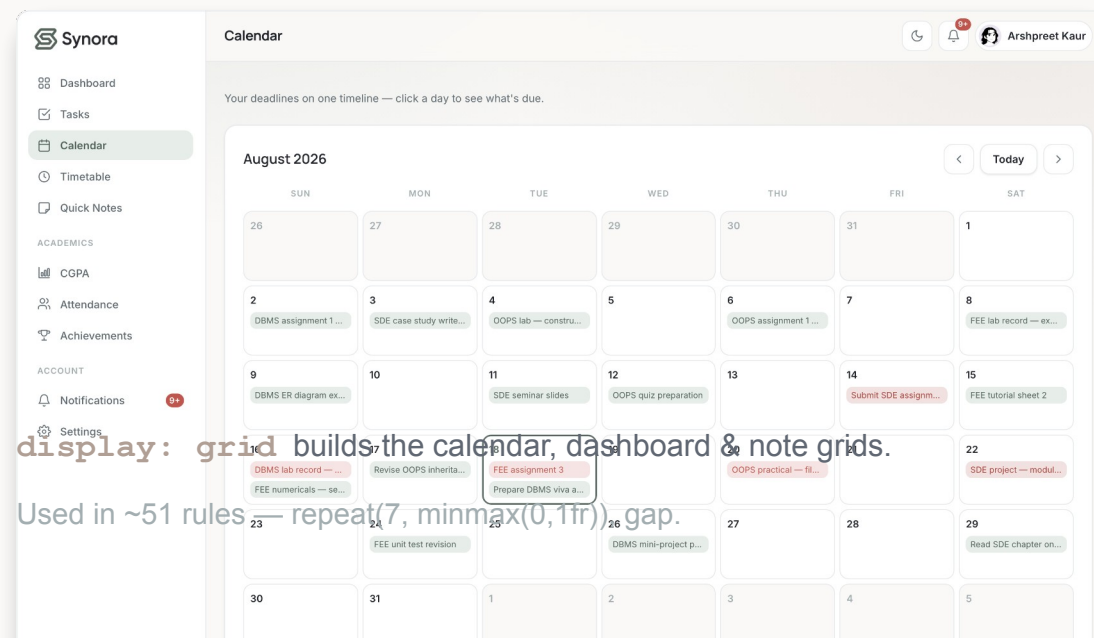
Flexbox — one-dimensional rows



display: flex aligns the top bar, toolbars and button rows.

Used in ~116 rules — align items, gap, flex-wrap.

Grid — two-dimensional layouts



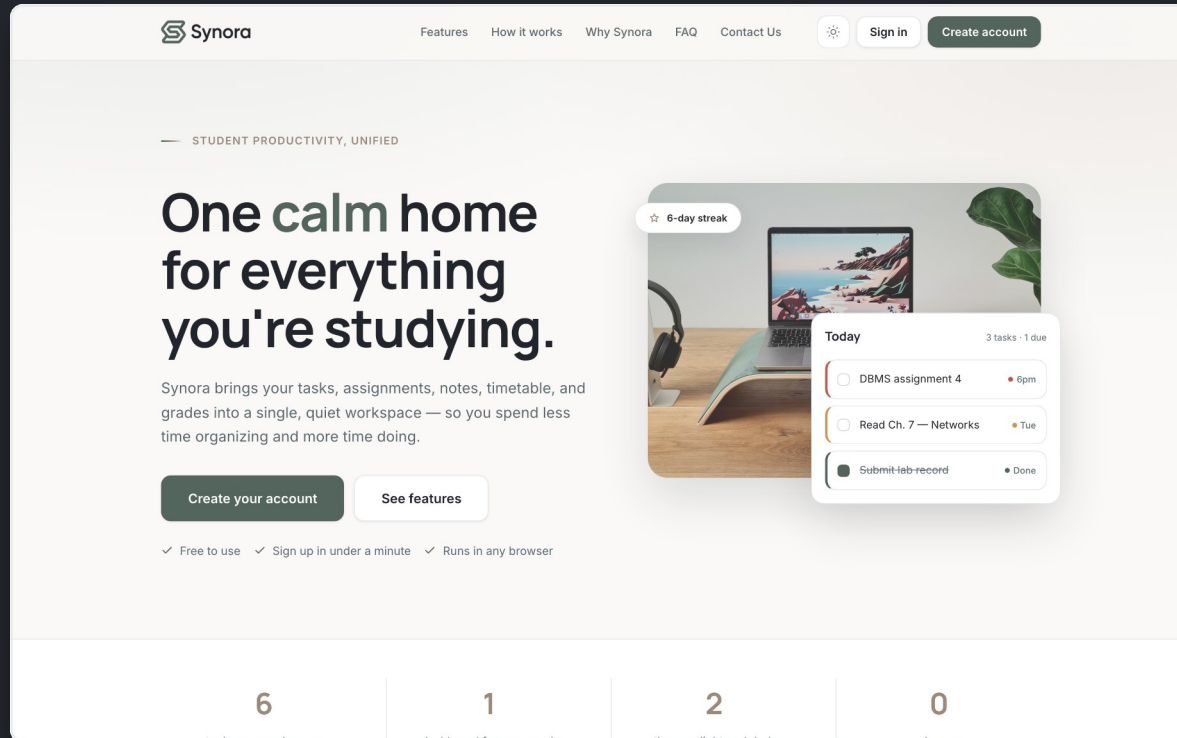
display: grid builds the calendar, dashboard & note grids.

Used in ~51 rules — repeat(7, minmax(0, 1fr)), gap.

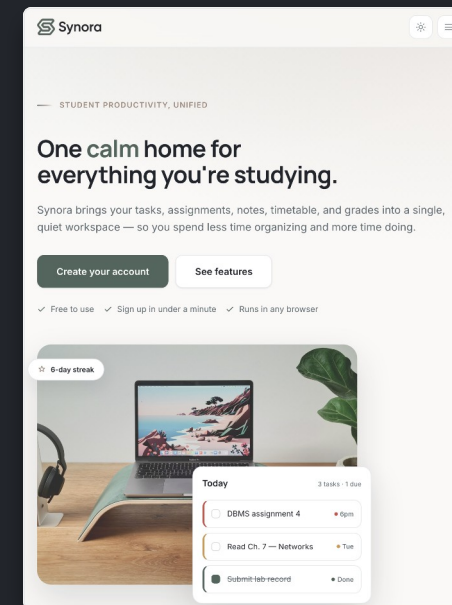
```
● ● ● styles/app.css — the seven-column calendar grid
```

```
.cal-grid { display: grid; grid-template-columns: repeat(7, minmax(0, 1fr)); gap: 6px; }
```

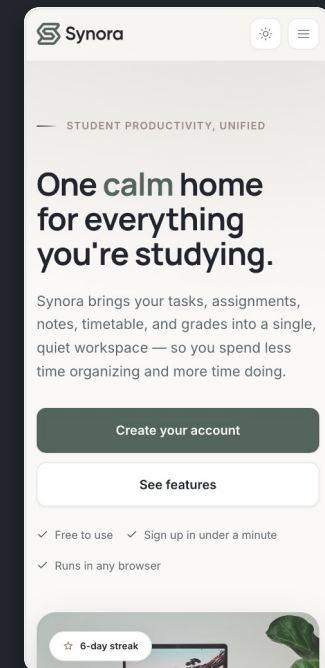
One page, every screen



1440px · Desktop



768px · Tablet



390px · Mobile

Desktop-first — Flexbox + Grid reflow, `max-width` media queries adapt down, and the nav collapses to a CSS-only menu.

From a static page to a product

HTML and CSS give structure and style; JavaScript adds the behaviour. The five fundamentals map directly onto real parts of Synora:

Variables	hold application state	<code>state = { view, sort, query }</code>
Functions	reusable operations	<code>render()</code> · <code>toggleComplete()</code>
Arrays	lists of things	<code>tasks</code> · <code>subjects</code> · <code>timetable</code>
Objects	structured records	<code>{ id, title, priority, deadline }</code>
Loops	render each item	<code>.map()</code> · <code>.filter()</code> · <code>.forEach()</code>

scripts/tasks.js — arrays, objects & loops

```
function filtered() {  
  var items = all().slice();  
  items = items.filter(function(t) {  
    return t.title.indexOf(q) >= 0;  
  });  
  items.sort(byDeadline);  
  return items;  
}
```

One function turns an array of task objects into the filtered, sorted list you see — variables, functions, arrays, objects and loops all in a few lines.

Making the interface dynamic

The page is a tree of nodes

document

```

└─ main.page
    └─ section.toolbar
        └─ input[data-search]
    └─ div[data-list]
        └─ div.tlist
            └─ div.trow ← one per task
  
```

SELECTORS ACTUALLY USED

`querySelector()` ×161

`querySelectorAll()` ×23

`getElementById()` ×9

Create, configure, insert — then remove

```

scripts/app.js — the toast lifecycle

var el = document.createElement("div"); // 1 create
el.className = "toast";
el.setAttribute("role", "status"); // 2 config
el.textContent = message;
region.appendChild(el); // 3 insert
...
el.remove(); // 4 delete
  
```

JS reads and writes the page through the DOM: select an element, change its text, classes or attributes, build new nodes, and remove them when done. classList is toggled in ~35 places.

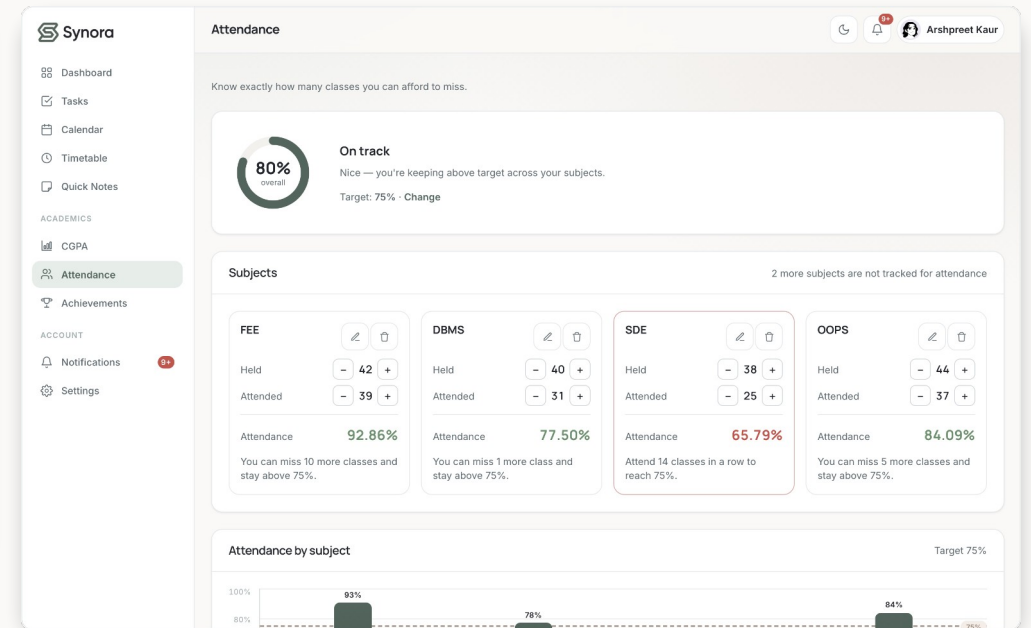
From a click to an updated screen



```
scripts/tasks.js - click & input listeners

var addBtn = document.querySelector("[data-add-task]");
addBtn.addEventListener("click", function() {
  openForm(null, render); // click → open the task form
});

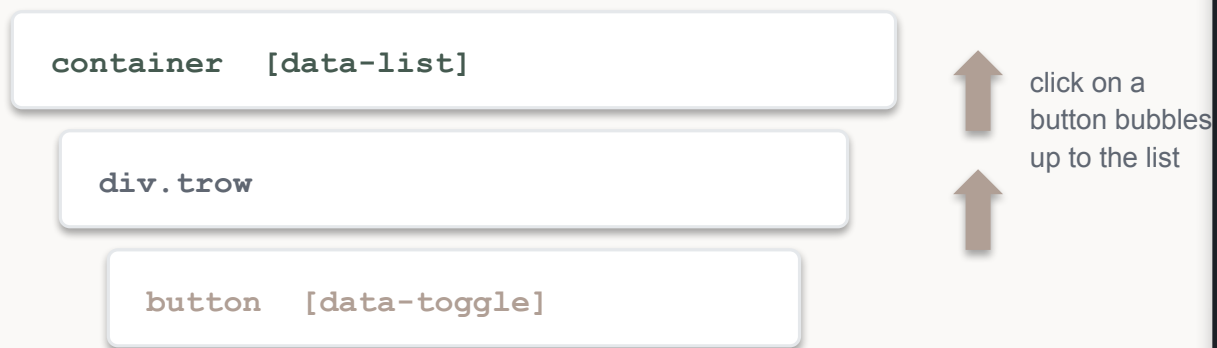
search.addEventListener("input", function(e) {
  state.query = e.target.value; render(); // each keystroke
});
```



Attendance +/- buttons fire click events that recompute every percentage instantly. `addEventListener` is used ~93×.

One listener, many buttons

Events bubble up the tree



```
scripts/tasks.js — event delegation
```

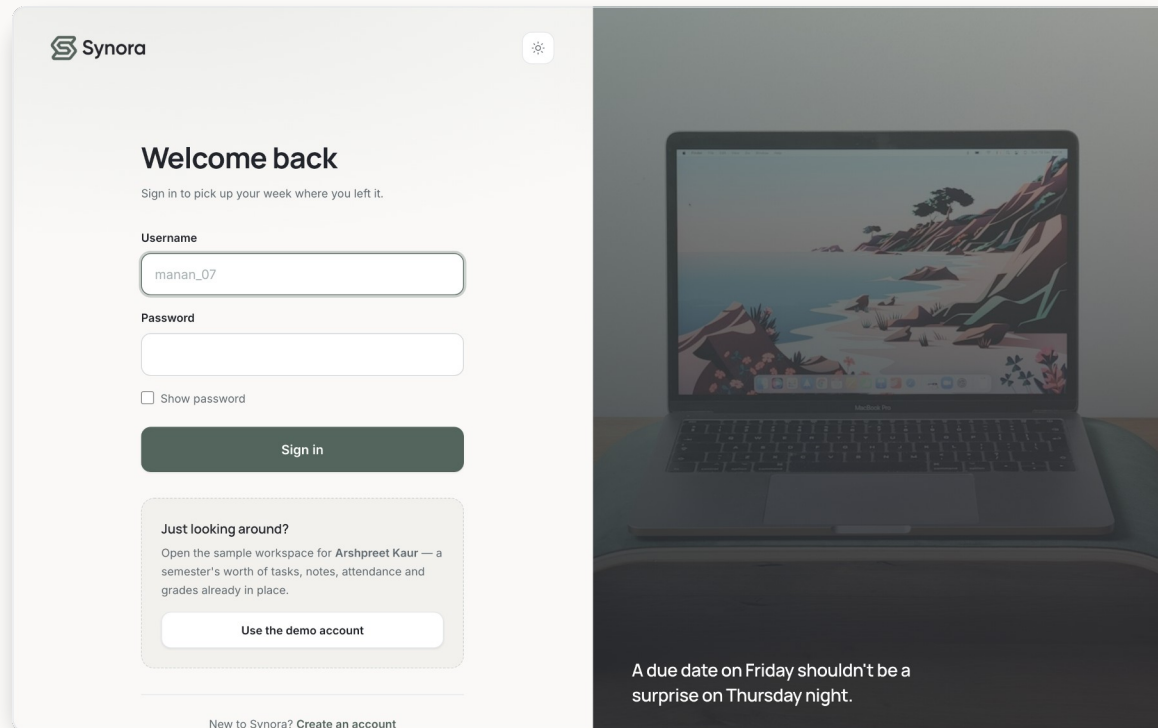
```
// ONE listener handles every task row
container.addEventListener("click", function(e) {
  var t = e.target.closest("[data-toggle]");
  var d = e.target.closest("[data-del]");
  if (t) { toggleComplete(t...); rerender(); }
  else if (d) { remove(d...); rerender(); }
});
```

IN SYNORA — HONESTLY

- **Event bubbling** used — clicks bubble to the parent
- **Event delegation** used — `.closest()` in ~28 handlers
- **preventDefault** used — forms & links (~11)
- **Explicit capturing** *concept only — not a pattern we use*

Task rows are re-rendered constantly, so binding a listener to each button would be fragile. Instead one listener sits on the container and uses `e.target.closest()` to find which button was clicked — that is event delegation, powered by bubbling.

Catching mistakes before they save



Sign-in and sign-up validate on submit and clear errors as you type.

Two layers of validation

HTML attributes

required, type=email, minlength — the browser's first check

JavaScript on submit

e.preventDefault(), trim, regex, then friendly inline errors

Live error clearing

the error disappears the moment the field is corrected

```
scripts/auth.js — sign-up validation

form.addEventListener("submit", function(e){
  e.preventDefault();           // stop native
  submit
  if (!EMAIL_RE.test(email))
    setError("email", "Check your email.");
});
```

Before you see Synora

Splash

logo, ring & wordmark
animate in



Handover

splash fades & lifts as the
page arrives underneath



Landing

the hero's staggered
entrance begins

HOW IT ACTUALLY WORKS

CSS @keyframes — spin, breathe, pop & sheen — pure CSS animation

A CSS transition — the splash fades + lifts on the .is-hidden class

JavaScript timing — shows for ≥ 2000 ms, waits for
DOMContentLoaded

DOM + state — toggles classes, then loader.remove() clears the node

Reduced motion — honoured — animation off, wait cut to 600ms

```
index.html — the reveal()
```

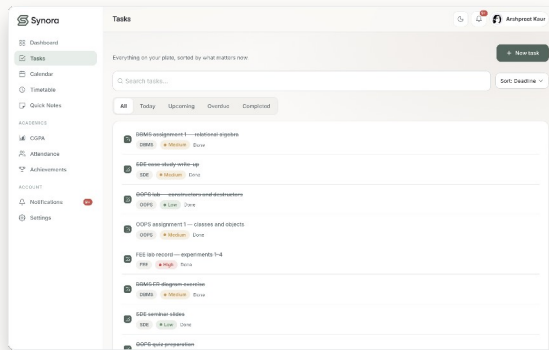
```
if (!ready || !timeUp) return;  
body.classList.add("is-ready");  
loader.classList.add("is-hidden");  
setTimeout(() => loader.remove(), 800);
```

CSS paints and animates; JavaScript only decides when the splash may leave.
The two movements overlap, so Synora feels like it opens rather than loads.

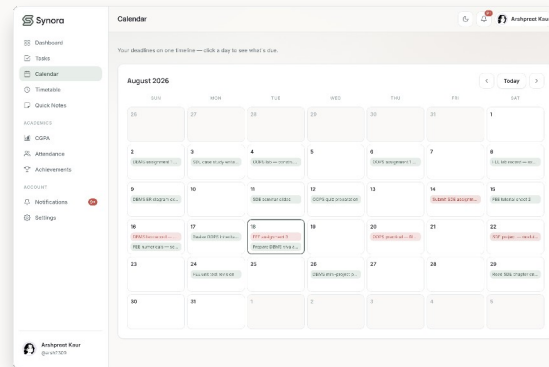
THE PRODUCT WE SHIPPED

Beyond the syllabus

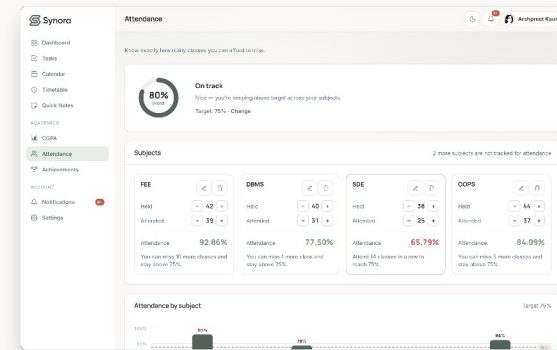
The syllabus gave us the parts — we built complete, connected features, plus personalization & avatars, light / dark themes, per-user workspaces and on-device persistence.



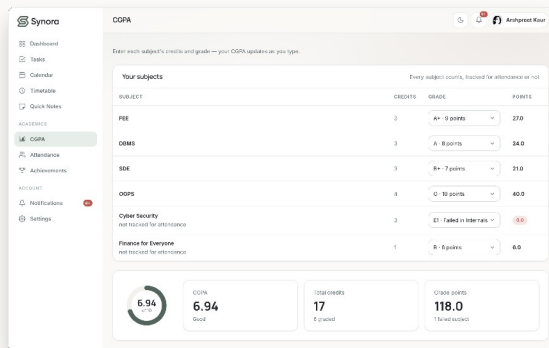
Task management



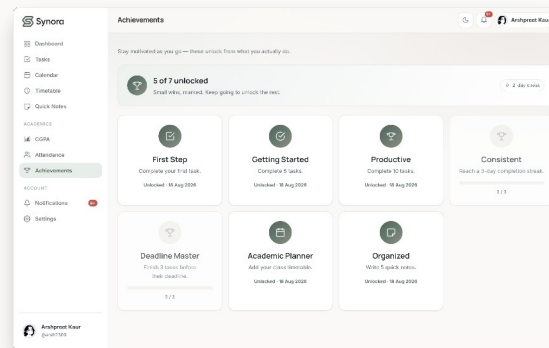
Smart calendar



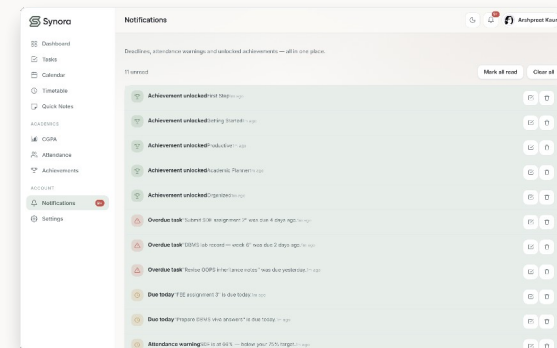
Attendance calculator



CGPA calculator



Achievements



Notifications

What we applied

HTML5	CSS3	JavaScript	DOM	Events
✓ Semantic structure ✓ Forms & inputs ✓ Accessibility basics ✓ Figure / media	✓ Box model ✓ Flexbox ✓ Grid ✓ Media queries	✓ Variables & functions ✓ Arrays & objects ✓ Loops ✓ localStorage	✓ querySelector(All) ✓ getElementById ✓ create / append ✓ remove nodes	✓ addEventListener ✓ Delegation ✓ Form validation ○ Capturing

✓ implemented in Synora ○ learned as a concept, not a primary pattern

THE TAKEAWAY

Concepts in, product out

SYLLABUS

HTML5

CSS3

JavaScript

DOM

Events



IMPLEMENTATION

Structure

Design system

Logic & state

Dynamic UI

Interaction



A real, responsive student-productivity product — built entirely from the syllabus, then taken further.

Synora — one calm home for everything you're studying.